

MATEMÁTICA

UNIDAD N°3

TRABAJO INTEGRADOR

3RA PARTE

MATEMÁTICA

TRABAJO PRÁCTICO INTEGRADOR - UNIDAD 3

Teoría de Conjuntos y Lógica aplicada: Modelado algorítmico con R

1. El Salto de la Abstracción al Dato

En la Unidad 2 modelamos hardware lógico; en esta Unidad 3, utilizaremos R y RStudio para gestionar estructuras de datos complejas. En el desarrollo de software, la Teoría de Conjuntos es la base fundamental de las consultas a Bases de Datos (SQL) y la manipulación de colecciones de objetos. En R, trabajaremos con la capacidad de procesar vectores como conjuntos matemáticos para resolver problemas de pertenencia, inclusión y partición.

2. Diccionario de Equivalencias de Conjuntos

Para este TP, traduciremos las operaciones de conjuntos y leyes lógicas a funciones de R que operan de forma nativa sobre vectores:

- **Unión:** Se usa `union(A, B)`. Combina elementos de ambos conjuntos eliminando duplicados.
- **Intersección:** Se usa `intersect(A, B)`. Selecciona únicamente los elementos presentes en ambos conjuntos.
- **Diferencia:** Se usa `setdiff(A, B)`. Devuelve elementos que están en A pero no en B.
- **Diferencia Simétrica:** Se debe derivar como la unión de las diferencias: `union(setdiff(A, B), setdiff(B, A))`.
- **Pertenencia:** Se usa el operador `%in%`. Devuelve un vector lógico indicando si un elemento existe en el conjunto.

3. Los Tres Desafíos Integradores

Desafío 1: Generador Algorítmico de Conjunto Potencia $P(A)$ **Contexto:** El crecimiento exponencial de los datos es un reto en programación. Un conjunto de n elementos genera 2^n subconjuntos. **Instrucciones:**

1. Crea un conjunto A con 4 elementos (por ejemplo: colores o números).
2. Utiliza funciones de combinación para generar todos los subconjuntos posibles de A.
3. Tu tarea: El script debe calcular automáticamente la longitud total del conjunto potencia y verificar mediante una sentencia lógica si el resultado es igual a 2 elevado a la cantidad de elementos de A.

MATEMÁTICA

4. Identifica y muestra por pantalla cuáles de esos subconjuntos son propios (aquellos que no son iguales al conjunto original).

Desafío 2: Verificador de Isomorfismo y Leyes de De Morgan Contexto: Debes demostrar empíricamente que las leyes de la lógica y los conjuntos son equivalentes. **Instrucciones:**

1. Define un Universo U del 1 al 10. Crea dos conjuntos A y B como subconjuntos de U.
2. Tu misión: Programar ambos lados de la Primera Ley de De Morgan: El complemento de (A unión B) es igual a (Complemento de A) intersección (Complemento de B).
3. Prueba obligatoria: Usa la función `setequal()` para comparar el resultado de la operación de la izquierda contra la de la derecha. El script debe devolver TRUE si la implementación es correcta.

Desafío 3: El "Data Splitter" y Validación de Particiones Contexto: En Ciencia de Datos, es crítico dividir un dataset en grupos que no se solapen (Particiones). **Instrucciones:**

1. Define un Universo D con 12 registros de prueba (pueden ser IDs del 1 al 12).
2. Crea tres subconjuntos P1, P2 y P3.
3. Tu misión: Diseña una función de validación que verifique las dos condiciones de una Partición:
 - Disjuntividad: La intersección entre cualquier par de subconjuntos debe ser vacía.
 - Recubrimiento: La unión de P1, P2 y P3 debe ser exactamente igual al Universo D.
4. El script debe arrojar un mensaje de "Partición Válida" o "Error en Partición" según corresponda.

4. ¿Qué se evalúa? (Rúbrica de éxito)

- **Rigor Técnico:** Uso correcto de funciones de conjuntos en R y manejo de vectores.
- **Pensamiento Algorítmico:** Capacidad para resolver la diferencia simétrica y el conjunto potencia mediante lógica de programación.
- **Validación:** Uso de `setequal()` o `identical()` para demostrar teoremas.
- **Documentación:** El script debe estar comentado explicando qué leyes de conjuntos (Idempotencia, De Morgan, etc.) justifican cada paso de la lógica.